

Sistema di Logistica Droni Resiliente con AI Agentica

Lorenzo Amorosa
Edoardo Buttazzi
Gregorio Cavulla

 Repository GitHub

Indice

1	Introduzione e Definizione del Servizio	3
1.1	Dominio Operativo	3
1.2	Stakeholder e Utenti Target	4
1.3	Analisi del Workload	4
1.4	Threat Model e Scenari di Guasto	5
2	Architettura del Sistema e Osservabilità	6
2.1	Design a Microservizi	6
2.2	Simulazione del Mondo Fisico (Digital Twin)	7
2.3	Osservabilità (Observability Strategy)	7
3	Agentic AI e Design MCP (Model Context Protocol)	9
3.1	Il Limite dell'Automazione Deterministica	9
3.2	Architettura MCP	9
3.3	Ruolo 1: The Health Agent	10
3.4	Ruolo 2: The Logistic Agent	10
3.5	Orchestrazione Dinamica	11
4	Operational Safety Policy e Agent Guardrails	12
4.1	Principio del Minimo Privilegio (Least Privilege)	12
4.2	Protezione contro Loop e Consumo Token	13
4.3	Protezione contro lo Scollegamento del Backend	13
4.4	Idempotenza delle Azioni	13
4.5	Human-In-The-Loop (HITL) ed Escalation	14
5	Meccanismi di Reliability e Scalabilità	15
5.1	Resilienza di Rete	15
5.2	Graceful Degradation del Server Centrale	15
5.3	Scalabilità Dinamica e Safety Limits Kubernetes	16
6	Chaos Engineering e Test di Fallimento	17
6.1	Metodologia	17
6.2	Esperimento 1: Primary Database Outage	17
6.3	Esperimento 2: Network Resilience (MQTT Broker Crash)	18
6.4	Esperimento 3: Load Spike e Scaling Agentico	18
6.5	Riepilogo Metriche	18
6.6	Lezioni Apprese	19

7	Analisi Economica: Costi, ROI e ROA (Return on Agent)	20
7.1	Service Level Objectives (SLO) e Costo del Downtime	20
7.2	Valutazione del Return on Agent (ROA) e Costi	21
7.2.1	Strategie di Ottimizzazione dei Costi Operativi (OPEX)	21
7.2.2	Mitigazione del Rischio e Preservazione del Capitale (CAPEX) . . .	21
7.3	Analisi dei Trade-Off	23
8	Conclusioni	24
8.1	Sintesi dei Risultati	24
8.2	Limitazioni Attuali	24
8.3	Sviluppi Futuri	25

Capitolo 1

Introduzione e Definizione del Servizio

1.1 Dominio Operativo

Il progetto si colloca all'intersezione tra due domini critici identificati nelle specifiche di corso: *Transportation* e *Industrial / IoT Monitoring*. Nello specifico, il sistema implementa un'architettura logistica autonoma per la gestione di una flotta di droni adibita alla consegna urbana di *payload* critici (forniture mediche d'emergenza, componentistica industriale ad alta priorità). In un servizio di tipo *high-stakes*, efficienza e affidabilità sono i requisiti fondamentali per la buona riuscita delle missioni.

Il Contesto Fisico e Simulato

Il sistema opera all'interno di un'area metropolitana modellata metricamente, con un raggio d'azione di 5000 m (5 km) attorno a un HUB logistico centrale posizionato alle coordinate base $[0.0, 0.0]$. L'ambiente operativo è altamente dinamico e soggetto a vincoli fisici realistici che l'Intelligenza Artificiale deve interpretare e gestire:

- **Gestione del Payload:** gli ordini generati presentano un peso variabile (da 0.5 kg a 5.0 kg) e tre livelli di priorità (*low*, *normal*, *high*).
- **Fisica del Volo:** i droni non sono entità ideali, ma agenti fisici il cui stato viene modellato in base allo sforzo. Il consumo della batteria e la velocità di crociera in consegna (variabile tra 50 e 80 km/h) sono calcolati dinamicamente in proporzione diretta al peso del carico trasportato; in fase di rientro senza carico il simulatore può raggiungere i 100 km/h. Ogni drone può trovarsi in uno dei quattro stati operativi: `IDLE`, `IN_DELIVERY`, `RETURNING`, `MAINTENANCE`.
- **Degrado Meccanico (Wear):** ogni missione accumula una percentuale di usura (da 0% a 100%) basata sulla distanza percorsa. Il superamento di soglie critiche richiede interventi di manutenzione preventiva per evitare danni critici.

1.2 Stakeholder e Utenti Target

Gli attori del sistema si dividono in due macro-categorie: gli *stakeholder* (focalizzati su gestione, sicurezza e sostenibilità) e gli *utenti target* (beneficiari diretti del servizio logistico).

Stakeholder

1. **Operatori di Flotta (Human-in-the-Loop):** Responsabili della sicurezza operativa. Tramite la dashboard web monitorano il sistema e agiscono da *gateway* decisionale, validando o respingendo le azioni ad alto impatto proposte dall'IA (es. scaling K8s oltre i limiti consentiti).
2. **SRE Team / DevOps:** Garantiscono l'alta disponibilità (*High Availability*) dell'infrastruttura. Monitorano i pod Kubernetes, il broker Mosquitto e InfluxDB, intervenendo su guasti hardware o di rete non risolvibili in autonomia dagli agenti.
3. **Business Owner e Controller Finanziari:** Supervisionano l'efficienza economica bilanciando CAPEX e OPEX. Controllano il consumo di token LLM (*budget ceiling*) e misurano il *Return on Agent* (ROA), ossia il risparmio generato dalla manutenzione predittiva dell'IA rispetto al costo delle chiamate API.

Utenti Target (Clienti del Servizio)

- **Entità Mittenti (Hub Medici, E-Commerce, Industrie):** Generano le richieste asincrone di consegna. Esigono il rigido rispetto dei *Service Level Objectives* (SLO), richiedendo la rapida evasione degli ordini *high priority* anche durante estremi picchi di carico (*load spikes*).
- **Destinatari Finali (Laboratori, Linee JIT):** Beneficiari ultimi del payload. In contesti critici o *Just-In-Time* (JIT), dipendono dalla totale resilienza del sistema, imponendo requisiti stringenti come downtime nullo ed eliminazione dei fallimenti logistici (es. schianti per batteria scarica).

1.3 Analisi del Workload

L'architettura è di tipo *event-driven* ed è sottoposta a un carico di lavoro ibrido, caratterizzato dalla coesistenza di flussi di dati continui (streaming IoT) ed eventi asincroni variabili (richieste di business e cicli di ragionamento AI). Per dimensionare correttamente i meccanismi di reliability e scalabilità, il workload è stato scomposto in tre componenti principali.

1. Telemetria IoT ad alta frequenza (Constant Load)

Il carico di base dell'infrastruttura è generato dalla flotta. Ogni Pod `drone_simulator` calcola costantemente il proprio movimento e il decadimento della batteria, trasmettendo un payload JSON contenente coordinate spaziali, stato operativo e usura. Ciascun drone pubblica un aggiornamento sul broker MQTT (topic `telemetry/drones`) con una

frequenza di un messaggio ogni 2 secondi. Questo genera un throughput in ingresso costante, che il `central_server` deve prima elaborare in RAM per mantenere aggiornata la dashboard real-time e successivamente persistere su InfluxDB strutturandolo come serie temporale.

2. Eventi di Business asincroni (Spiky Load)

Il secondo vettore è rappresentato dalle richieste di consegna generate dai nodi cliente (`client_simulator.py`). Il traffico è composto da ordini generati casualmente e pubblicati sul topic `business/ordini/nuovi`. A questo si sovrappongono i picchi anomali generati negli scenari di stress testing, pensati per forzare l'intervento reattivo del sistema.

3. Carico di Inferenza e Polling Agentico (Read-Heavy Spikes)

Una peculiarità delle architetture che sfruttano AI Agentica è il carico indiretto generato dall'Intelligenza Artificiale stessa sull'infrastruttura sottostante. L'orchestratore centrale (`logistic_ai_brain.py`) risveglia periodicamente gli agenti e ogni risveglio scatena diverse operazioni in lettura (*polling*):

- interrogazione intensiva di InfluxDB per estrarre la finestra temporale (*time-window*) degli ultimi 5–60 minuti di telemetria e ordini;
- chiamate alle API di Kubernetes per ottenere lo stato dei Pod attivi in tempo reale.

Questo workload *a raffiche* richiede che il database e le API di Kubernetes rispondano con latenze minime, poiché un ritardo nella fase di *information gathering* prolungherebbe i tempi di inferenza del Large Language Model, violando gli SLO di reattività del sistema.

1.4 Threat Model e Scenari di Guasto

A completamento dell'analisi del workload, sono stati identificati gli scenari di guasto considerati come baseline architetturale del progetto. Questi scenari guidano le scelte di reliability discusse nel Capitolo 5 e vengono validati empiricamente nel Capitolo 6.

- **Caduta del broker MQTT (mosquitto):** interruzione del canale di scambio eventi tra droni, server centrale e simulatori client. È mitigata dalla rischedulazione nativa di Kubernetes e dal meccanismo di *exponential backoff* del client `paho-mqtt`.
- **Indisponibilità del database primario (influxdb):** perdita temporanea dello storage delle serie temporali. È mitigata dal pattern di *Graceful Degradation* del `central_server`, che preserva lo stato in-memory e neutralizza l'eccezione di scrittura per non bloccare le code real-time.
- **Picchi anomali di carico (load spike):** immissione massiva di ordini in pochi secondi. È gestita dal layer agentico tramite scaling dinamico del Deployment `drone-simulator`, vincolato dalle policy di sicurezza descritte nel Capitolo 4.
- **Allucinazioni e loop dell'LLM:** comportamenti non deterministici degli agenti che potrebbero generare cicli infiniti di tool calling o azioni distruttive. Sono contenuti dal layer MCP (read-only su telemetria, assenza di tool distruttivi) e dai contatori di iterazione descritti nel Capitolo 4.

Capitolo 2

Architettura del Sistema e Osservabilità

L'architettura del progetto è basata su Kind (*Kubernetes IN Docker*), utilizzato per avviare cluster Kubernetes locali e leggeri all'interno di container Docker. Questa scelta consente di replicare molti comportamenti di un cluster reale (scheduler, networking, controller) in un ambiente contenuto e facilmente ricreabile, ideale per validazione e test prima di un eventuale rilascio in produzione.

Kubernetes è un orchestratore di container che gestisce deployment, scaling e resilienza delle applicazioni: organizza i container in **Pod**, mantiene lo stato desiderato tramite **Deployment** e **ReplicaSet**, espone i servizi con **Service** e fornisce funzionalità native di autoscaling e self-healing. Tutte queste funzionalità sono richiamate nel progetto tramite l'invocazione delle API di `kubectl`.

Docker è la piattaforma di containerizzazione che confeziona applicazioni e dipendenze in immagini portabili, garantendo isolamento, avvisi rapidi e coerenza tra ambienti. Funge da base su cui costruire e distribuire le immagini eseguite da Kind (e quindi da Kubernetes).

2.1 Design a Microservizi

Il progetto adotta un design a microservizi che separa chiaramente responsabilità e domini di failure in tre blocchi indipendenti: il **Data Layer** (InfluxDB), dedicato alla persistenza di metriche e serie temporali; il **Message Broker** (MQTT/Mosquitto), che funge da canale asincrono di scambio eventi; e il **Logic Layer** (server centrale e servizi applicativi), che implementa l'elaborazione e l'esposizione delle funzionalità di business.

Questa separazione è anche fisica: ogni blocco viene eseguito in Pod distinti, con **Service** dedicati per la discovery via DNS interno (es. `influxdb-service` e `mosquitto-service`), in modo che il layer logico non dipenda da IP statici ma soltanto da endpoint di servizio. Il broker disaccoppia produttori e consumatori: il layer logico può scalare o riavviarsi senza spezzare l'ingest, mentre InfluxDB resta un componente *stateful* con volume persistente (PVC) e una strategia di avvio che privilegia la consistenza dei dati.

Gli script Python non vengono eseguiti come processi manuali sui nodi: Kubernetes li avvia come processo principale del container tramite il `command` del Deployment (un container = un servizio, oppure più container nello stesso Pod quando serve co-localizzazione, ad esempio per condividere dati/volumi e ridurre la latenza). Lo scheduler decide su quale nodo eseguire ciascun Pod; il campo `replicas` permette di distribuire più istanze

di uno stesso microservizio su nodi diversi per aumentare throughput e tolleranza ai guasti, mantenendo invariata l'interfaccia grazie ai **Service**.

2.2 Simulazione del Mondo Fisico (Digital Twin)

I digital twin di droni e client sono implementati come generatori controllati di telemetria e domanda, utili a validare l'architettura end-to-end senza dipendere da dispositivi fisici.

Il twin del drone (in `drone_simulator.py`) mantiene uno stato interno minimale ma realistico (posizione, batteria, usura, stato operativo) che evolve a tick: pubblica periodicamente la telemetria sul broker MQTT (topic `telemetry/drones`) e si mette in ascolto di comandi sul proprio canale dedicato `comandi/<DRONE_ID>`, reagendo a eventi come l'assegnazione di una missione o il rientro alla base e simulando consumi e degrado fino alla condizione di **MAINTENANCE**.

In parallelo, il twin del client (in `client_simulator.py`) genera carico di business pubblicando nuovi ordini su MQTT (topic `business/ordini/nuovi`) con intervalli casuali e payload strutturati (coordinate di pickup e drop, peso, priorità), così da stressare broker e layer applicativo in modo riproducibile.

Dal punto di vista infrastrutturale, entrambi vengono eseguiti su Kubernetes come Deployment separati, ciascuno nel proprio Pod (configurati nei manifest `configs/drone.yaml` e `configs/client.yaml`): il processo principale del container è lo script Python definito nel `command` del manifest, mentre la connettività verso l'infrastruttura è ottenuta via service discovery (`mosquitto-service`).

La moltiplicazione della flotta è ottenuta in modo nativo tramite lo scaling: aumentando il valore di `replicas` del Deployment `drone-simulator`, Kubernetes crea più Pod identici e, poiché `DRONE_ID` viene derivato dal `metadata.name` del Pod, ogni replica corrisponde a un nuovo drone digitale con identità e canale comandi distinti. Lo scaling può inoltre essere attivato in modo automatico dall'agente: il server MCP espone il tool `scale_drone_deployment`, che invoca la patch della risorsa `deployment/scale` (equivalente concettuale di un `kubectl scale`), con i guardrail operativi descritti nel Capitolo 4 per mantenere sotto controllo l'impatto delle azioni automatiche sul cluster.

2.3 Osservabilità (Observability Strategy)

Un sistema distribuito deve permettere una chiara visibilità operativa per consentire il monitoraggio dello stato, la tracciabilità delle decisioni agentiche e la rapida mitigazione dei guasti. La strategia di osservabilità adottata si fonda su tre pilastri: la visualizzazione real-time tramite una dashboard web centralizzata, il tracciamento di log strutturati per garantire l'auditability delle azioni e la persistenza di metriche su serie temporali.

1. Dashboard Web Centralizzata

La visibilità generale dello stato del sistema è affidata al server centrale (`central_server.py`), che ospita un'interfaccia utente web integrata tramite template HTML asincroni e un set di endpoint REST nativi in Flask:

- `/api/status`: fornisce uno snapshot numerico aggregato del sistema, con conteggio dei droni attivi, degli ordini pendenti e delle consegne completate, oltre alla configurazione dei nodi di rete (broker MQTT e istanza InfluxDB);

- `/api/orders`: espone la coda dei messaggi in stato `PENDING` memorizzati in RAM;
- `/api/drones`: restituisce la matrice completa dei droni agganciati al cluster, visualizzandone dinamicamente lo stato e le relative metriche;
- `/api/completed`: elenca lo storico logistico delle consegne andate a buon fine.

Lato client, la dashboard esegue un ciclo di polling asincrono tramite chiamate `fetch()` JavaScript ogni 5000 ms (5 secondi). Questo intervallo garantisce un aggiornamento visivo costante senza saturare il server.

2. Log di Sistema e Audit Trail dei Comandi Agentici

Il tracciamento dei log è strutturato su più livelli per garantire l'ispezionabilità (*auditability*) richiesta dal dominio:

1. **Registro di Audit Strutturato** (`audit_actions.jsonl`): gestito nativamente dal modulo `drone_mcp_layer.py`, archivia in formato JSON Lines ogni singola operazione di scrittura o di rimediazione avviata dall'IA. Per ciascuna azione vengono tracciati il timestamp, il tipo di tool invocato e l'esatto payload utilizzato.
2. **Registro delle Approvazioni Umane** (`pending_approvals.jsonl`): traccia il ciclo di vita delle richieste degli agenti quando viene superato un confine di policy. Il file contiene la richiesta iniziale dell'agente, la giustificazione logica fornita dall'LLM e il successivo cambiamento di stato causato dall'operatore.
3. **Log di Runtime e tracciamento dei costi**: i simulatori dei droni stampano a terminale log continui sul proprio stato. In parallelo, l'orchestratore e gli agenti loggano per ciascuna iterazione l'esatto consumo di token restituito dai modelli, metrica cruciale per la verifica dei tetti di spesa discussi nel Capitolo 7.

3. Metriche di Serie Temporalì su InfluxDB

L'ultimo pilastro dell'osservabilità riguarda l'archiviazione persistente dei dati operativi sotto forma di serie temporali. Quando il server centrale intercetta i messaggi sui topic MQTT, scrive in modalità sincrona record all'interno del bucket `iot_data`:

- **Measurement `drone_telemetry`**: registra l'evoluzione fisica della flotta;
- **Measurement `business_orders`**: monitora il flusso di ordini in ingresso.

Tramite il layer MCP (in particolare i tool `get_drones_telemetry` e `get_pending_orders`), gli LLM possono analizzare finestre temporali passate (*time-windows* di 5 o 60 minuti) per estrarre trend di carico o pattern di deterioramento hardware, base informativa per le decisioni di scaling e manutenzione.

Capitolo 3

Agentic AI e Design MCP (Model Context Protocol)

3.1 Il Limite dell'Automazione Deterministica

Tradizionalmente l'assegnazione degli ordini ai vettori sarebbe gestita con un approccio deterministico basato su alberi decisionali (scripting *if-else*). Sebbene questo metodo sia efficiente per carichi di lavoro lineari, si rivela inadeguato nel dominio complesso affrontato dal sistema.

Nel digital twin descritto nel Capitolo 2, ogni missione è influenzata da una combinazione di variabili eterogenee: la priorità dell'ordine, il peso del carico, la percentuale di batteria residua, il tasso di usura del mezzo, la distanza euclidea dall'HUB. Scrivere regole rigide per coprire ogni combinazione porterebbe a un codice fragile e difficilmente manutenibile.

Gli agenti superano questo limite sostituendo la rigidità deterministica con un ragionamento probabilistico: valutano dinamicamente i trade-off operativi e ottimizzano il matching ordine-drone in tempo reale, adattandosi a contesti imprevedibili senza necessità di pre-programmazione esplicita.

3.2 Architettura MCP

Per garantire che l'IA operi all'interno di confini sicuri, il sistema implementa un'architettura basata sul *Model Context Protocol* (MCP). L'MCP funge da strato di interposizione tra i loop di ragionamento degli LLM (confinati in `logistic_ai_brain.py`) e l'infrastruttura fisica simulata su Kubernetes.

L'LLM non possiede credenziali dirette né permessi di root sui database: comunica esclusivamente tramite richieste HTTP al server MCP, il quale espone due categorie di *tool*:

- **Observer Tools (Read-Only):** strumenti sicuri utilizzati dall'agente per la fase di raccolta delle informazioni. Includono `get_drones_status` (interrogazione API Kubernetes), `get_drones_telemetry` e `get_pending_orders` (estrazione dati in time-window da InfluxDB).
- **Remediation Tools (Write):** strumenti attivi che modificano lo stato del sistema. Includono `send_mqtt_command` per l'assegnazione delle missioni e `scale_drone_deployment`

per l'auto-scaling dei Pod. Essendo potenzialmente impattanti, la loro esecuzione nel modulo `drone_mcp_layer.py` è subordinata alle policy descritte nel Capitolo 4.

3.3 Ruolo 1: The Health Agent

L'**HealthAgent** si occupa della resilienza dell'infrastruttura, della diagnostica della flotta e del dimensionamento delle risorse fisiche. Opera analizzando i flussi di telemetria e agisce come supervisore automatizzato della salute del sistema, gestendo monitoraggio, triage dell'usura e auto-scaling.

Compiti principali

L'agente analizza in modo proattivo il rapporto tra gli ordini pendenti e la capacità attuale della flotta. Se rileva un collo di bottiglia, decide a quante unità (e quindi Pod Kubernetes) scalare il cluster per mantenere inalterati gli SLO. In alcuni casi, descritti nel Capitolo 4, richiede direttamente l'intervento umano.

Il ciclo di ragionamento dell'agente include anche l'identificazione dei guasti (es. esaurimento improvviso della batteria in volo o schianti). Se un drone si trova in stato **MAINTENANCE** ma le sue coordinate spaziali differiscono significativamente da quelle dell'HUB centrale $[0.0, 0.0]$, l'**HealthAgent** deduce autonomamente che il mezzo è precipitato sul campo e innesca le procedure di recupero previste.

3.4 Ruolo 2: The Logistic Agent

Il **LogisticAgent** si occupa della logica di business del sistema, gestendo la coda degli ordini asincroni inviati dai client simulator e coordinando l'effettiva assegnazione delle missioni ai singoli mezzi.

Compiti principali

L'agente logistico incrocia i dati degli ordini con lo stato telemetrico real-time dei droni in stato **IDLE**. L'assegnazione non avviene secondo criteri puramente sequenziali (FIFO), bensì attraverso una ponderazione del rischio:

1. **Analisi dell'ordine:** il peso del pacco (`weight_kg`) influisce direttamente sulle prestazioni del drone: un pacco pesante riduce la velocità di crociera e incrementa il consumo di batteria.
2. **Contesto spaziale:** la distanza euclidea impone un calcolo preventivo dell'autonomia necessaria al viaggio.
3. **Stato del mezzo:** il livello di batteria attuale e il grado di usura meccanica (*wear*) accumulato.

Se un ordine presenta priorità elevata (*high priority*) ma un peso prossimo al limite massimo di 5.0 kg, l'agente eviterà di assegnarlo a droni con usura elevata ($> 70\%$) o con batteria insufficiente, riducendo le probabilità di malfunzionamenti. Una volta individuata la coppia ottimale ordine-drone, l'agente invoca il tool MCP `send_mqtt_command` che scatena la partenza della missione.

3.5 Orchestrazione Dinamica

Il coordinamento e l'attivazione degli agenti sono gestiti dall'orchestratore centrale (`logistic_ai_brain.py`). Per massimizzare l'efficienza economica (*Budget Compliance*, Capitolo 7), l'orchestratore adotta un'architettura reattiva basata sul contesto, articolata in due meccanismi chiave.

1. **Triage Manager e Context Injection:** gli agenti non sprecano interazioni inutili. È l'orchestratore ad agire da router (`triage_manager`): analizza la fotografia del sistema e decide se serve l'intervento dell'`HealthAgent`, del `LogisticAgent` o di entrambi. Una volta stabilito il target, filtra i dati (fornendo, ad esempio, al `LogisticAgent` solo i droni IDLE) e li inietta direttamente nel prompt iniziale dell'agente (*Context Injection*). Questo costringe l'LLM a passare immediatamente alla fase di invocazione dei tool di rimediazione, evitando passaggi di ragionamento ridondanti.
2. **Esecuzione parallela asincrona:** qualora il *Triage Manager* stabilisca che sia necessaria sia un'azione di manutenzione/scaling sia un'assegnazione di ordini, i due agenti non vengono eseguiti in modo sequenziale. L'orchestratore li istanzia e li avvia in parallelo su processi separati (`threading.Thread`), facendoli successivamente ricongiungere. Questo approccio asincrono dimezza la latenza del ciclo decisionale, permettendo al sistema di rispondere in modo più reattivo a picchi di traffico imprevisti.

Capitolo 4

Operational Safety Policy e Agent Guardrails

L'introduzione di layer agentici come MCP espone il sistema a rischi intrinseci legati ad allucinazioni, latenze e comportamenti non deterministici. Per mitigare tali rischi sono state introdotte specifiche *Safety Policy* e vincoli architetturali, pensati per garantire che ogni azione degli agenti rimanga verificabile, confinata e sicura.

4.1 Principio del Minimo Privilegio (Least Privilege)

In linea con le specifiche di progetto, l'architettura applica il principio del *Least Privilege*: l'Intelligenza Artificiale è utilizzata come *assistente operativo* e non può in alcun caso avere il controllo illimitato dell'infrastruttura.

Per implementare questo livello di sicurezza, i cicli di ragionamento dei due agenti sono confinati all'interno di un ambiente di esecuzione isolato gestito dall'orchestratore. Gli agenti non possiedono credenziali di root, né token diretti al cluster Kubernetes o al database InfluxDB. L'unica interfaccia verso il mondo reale è il server MCP, che agisce come *chokepoint* di sicurezza applicando due regole fondamentali:

- **Accesso Read-Only alla telemetria:** la telemetria operativa viene esposta esclusivamente in modalità di sola lettura. Gli strumenti di osservazione implementano sotto il cofano solo `query_api` pre-compilate in Python. L'agente non ha facoltà di comporre query *Flux* o *SQL* arbitrarie, annullando matematicamente il rischio di *injection*.
- **Divieto assoluto di azioni distruttive:** non è stato esposto alcun tool che permetta la compromissione dei dati primari. Azioni ad alto rischio come il **DROP** di un bucket, il **DELETE** di un ordine storico su InfluxDB o la terminazione dei processi core sono fisicamente impossibili per l'agente. Il codice del layer MCP (`drone_mcp_layer.py`) omette intenzionalmente qualsiasi implementazione logica di questi comandi.

Si crea così un perimetro di sicurezza per cui, anche nel caso di un'allucinazione logica in cui l'LLM tentasse autonomamente di cancellare dati operativi o distruggere il cluster, le richieste verrebbero rigettate per inesistenza del metodo, preservando l'integrità del sistema.

4.2 Protezione contro Loop e Consumo Token

Una vulnerabilità comune degli agenti è la possibilità che entrino in un loop di ragionamento infinito. Questo accade quando il modello subisce un'allucinazione e continua a invocare le stesse chiamate API senza giungere a una conclusione. Oltre a degradare il sistema, ciò porterebbe rapidamente all'esaurimento del budget di token.

L'**HealthAgent** dispone perciò di un contatore che traccia le chiamate effettuate nello stesso turno, impostato a 3 iterazioni: se l'agente non riesce a completare il ragionamento entro tale limite, il processo viene terminato forzatamente e viene restituito un messaggio predefinito. È inoltre presente un secondo controllo sul formato della risposta: se l'agente produce per due volte consecutive un output che non rispetta lo schema atteso, viene bloccato allo stesso modo. Questi due meccanismi limitano l'ingresso in cicli infiniti, mantenendo la spesa entro il budget stabilito.

4.3 Protezione contro lo Scollegamento del Backend

È stato implementato un interruttore di sicurezza per impedire lo spam di chiamate API verso l'LLM quando l'infrastruttura di backend va offline. Se il server MCP si disconnette, la funzione di recupero dello stato globale fallisce sistematicamente: senza protezioni, il loop continuerebbe a girare ogni pochi secondi inviando agli agenti contesti vuoti o dati obsoleti, costringendoli a prendere decisioni basate sul nulla e generando centinaia di invocazioni API a vuoto.

Per evitare questo spreco di token e risorse, ogni chiamata fallita verso il server incrementa un contatore dedicato. Al raggiungimento di una soglia prefissata scatta automaticamente un *kill-switch* che congela l'attività degli agenti. Una volta attivata la sospensione globale, il sistema effettua un controllo periodico dello stato del backend; non appena il server MCP torna online, il contatore viene azzerato, l'interruttore di sicurezza si riarma e il flusso operativo riprende autonomamente la sua normale esecuzione.

4.4 Idempotenza delle Azioni

In architetture di questo tipo, l'idempotenza è una proprietà di sicurezza fondamentale per garantire la robustezza del sistema e prevenire fallimenti catastrofici causati da allucinazioni o comportamenti ripetitivi degli LLM. Un'operazione è idempotente quando il suo effetto sullo stato del sistema è lo stesso, sia che venga eseguita una sola volta sia che venga eseguita più volte.

L'esempio più significativo di questa protezione è il meccanismo di scaling della flotta. Quando l'**HealthAgent** determina la necessità di espandere il numero di droni e invoca `scale_drone_deployment`, il codice in `drone_mcp_layer.py` esegue un'operazione di *patching* sullo stato del cluster tramite la funzione `patch_namespaced_deployment_scale`. Questa scelta garantisce l'idempotenza grazie a due fattori:

- **Logica di stato desiderato (dichiarativa):** il tool accetta come parametro il numero di repliche desiderate (ad esempio 6). Se l'LLM entra in un loop e ripete cinque volte lo stesso comando, Kubernetes riceverà cinque richieste consecutive che

dichiarano lo stesso stato target (6 Pod); il control-plane verificherà che il Deployment ha già raggiunto la quota di 6 repliche e ignorerà completamente le successive quattro chiamate, senza eseguire alcuna allocazione extra.

- **Prevenzione del collasso per esaurimento risorse:** se il tool MCP fosse stato progettato in modo *incrementale* (aggiungendo un numero di droni a ogni chiamata), cinque chiamate consecutive dettate da un'allucinazione avrebbero inserito decine di droni fantasma nel cluster, saturando istantaneamente le risorse computazionali dei nodi, congestionando la rete MQTT e violando il tetto massimo di spesa.

4.5 Human-In-The-Loop (HITL) ed Escalation

Il paradigma *Human-In-The-Loop* (HITL) rappresenta l'ultimo livello di difesa contro allucinazioni o decisioni economicamente dannose generate autonomamente dall'Intelligenza Artificiale. Nel sistema, l'automazione pura viene interrotta non appena un'azione degli agenti supera i confini di sicurezza prestabiliti dalle policy, forzando un'escalation verso un supervisore umano.

La regola architetturale dei 6 droni

All'interno del modulo `drone_mcp_layer.py`, la costante `POLICY_LIMITS` fissa a **6** il numero massimo di droni allocabili in totale autonomia (`max_drones_auto`). Il flusso operativo è regolato dalla seguente policy:

- **Sotto la soglia (≤ 6):** l'`HealthAgent` ha pieno controllo dichiarativo e può invocare direttamente `scale_drone_deployment` per adattare il cluster Kubernetes ai picchi di carico.
- **Sopra la soglia (> 6):** qualora un workload maggiore richieda un dimensionamento superiore a 6 Pod, il layer MCP blocca l'esecuzione deterministica del comando considerandolo ad alto impatto economico. L'agente è obbligato a interrompere il proprio loop e a invocare lo strumento `request_human_approval`.

Il portale di approvazione

L'intercettazione e la gestione delle escalation sono demandate a un servizio Flask indipendente (`human_approval_manager.py`) in ascolto sulla porta 5002, che espone una dashboard web dedicata agli operatori umani.

Quando l'agente richiede un'escalation, il server MCP persiste la richiesta nel file `pending_approvals.jsonl`. La dashboard legge questo registro e renderizza una scheda informativa per l'operatore. Ciascuna scheda mostra in modo trasparente l'identificativo univoco (`request_id`), l'azione richiesta, il payload JSON (es. `{"replicas": 8}`) e, soprattutto, la giustificazione logica in linguaggio naturale fornita dall'LLM (`reason`).

L'operatore dispone di un diritto di veto assoluto tramite due interfacce: **Approve** convalida la transizione ed esegue istantaneamente il patch su Kubernetes impostando il parametro `force=True`, scavalcando i blocchi preventivi del codice; **Deny** contrassegna la richiesta come `rejected` e, al ciclo successivo, l'`HealthAgent` manterrà lo stato della flotta inalterato.

Capitolo 5

Meccanismi di Reliability e Scalabilità

Il presente capitolo illustra le soluzioni applicate per gestire i temporanei disservizi e per scalare le risorse al bisogno. L'architettura fa ampio uso dei comportamenti *out-of-the-box* forniti dalle librerie di base, arricchiti da pattern architetturali pensati per confinare gli errori e rendere il sistema reattivo al traffico.

5.1 Resilienza di Rete

Nei sistemi distribuiti basati su messaggistica (come i client implementati in `drone_simulator.py`), la sconnessione temporanea del broker MQTT è un'evenienza da gestire nativamente.

In fase di avvio (funzione `run()` del simulatore), i droni eseguono un ciclo di retry custom: qualora il broker non sia ancora schedulato (ad esempio durante un deployment asincrono in un cluster Kubernetes), la connessione fallisce e viene avviato un poller ritardato di 5 secondi tramite `time.sleep`, ripetuto fino al successo dell'handshake.

A regime, la maggior parte della resilienza di rete è demandata ai meccanismi interni della libreria `paho-mqtt` e al suo metodo `loop_start()`. In caso di caduta momentanea del servizio Mosquitto, il client Python intercetta il blocco della socket e avvia autonomamente una strategia di *exponential backoff* per i tentativi di riconnessione, ripristinando la sottoscrizione ai topic operativi (`comandi/#`) senza introdurre logiche custom che rischierebbero di sovraccaricare il thread principale.

5.2 Graceful Degradation del Server Centrale

Nel sistema, `central_server.py` gestisce sia la persistenza delle telemetrie su InfluxDB sia l'esposizione della dashboard Flask in tempo reale per la supervisione della flotta. Per evitare che un downtime del database causi il collasso del monitoraggio live, è stato adottato un approccio ibrido con degradazione controllata.

Il pattern prevede un *dual-write* all'arrivo di ogni messaggio MQTT (handler `on_message`), strutturato secondo la seguente logica:

```
1 def on_message(client, userdata, msg):
2     # 1. Aggiornamento immediato dello stato in-memory (per la
3     dashboard live)
4     state["drones"][drone_id] = parsing(msg.payload)
```



```

4      # 2. Scrittura persistente su InfluxDB isolata da eccezioni
5      try:
6          write_api.write(bucket, org, record)
7      except InfluxDBError as e:
8          # L'eccezione viene neutralizzata per preservare i
9      servizi realtime
10         log_warning("InfluxDB non raggiungibile. Degradazione del
            servizio log storici.")

```

Listing 5.1: Schema logico del meccanismo di Graceful Degradation nel server centrale.

Se il container InfluxDB risulta non accessibile, l'eccezione viene neutralizzata nel blocco `try/except` interno all'handler. Di conseguenza, il sistema perde il salvataggio dei log storici (degradazione del servizio) ma mantiene perfettamente operativi il tracciamento real-time dei pacchetti e i cruscotti per gli operatori (comportamento *graceful*), isolando il guasto al solo storage layer senza bloccare la coda delle missioni.

5.3 Scalabilità Dinamica e Safety Limits Kubernetes

L'architettura supera lo scaling passivo (basato unicamente su carico CPU/RAM) delegando la valutazione dei bisogni logistici agli agenti AI dotati di capacità MCP.

In particolare, il `LogisticAgent` e l'`HealthAgent` dispongono di un tool per richiedere dinamicamente più droni e smaltire la coda (`pending_orders`). Internamente, questo tool richiama la funzione `scale_drone_deployment` in `drone_mcp_layer.py`, che interagisce via API HTTP con il control-plane di Kubernetes. Sfruttando la libreria client ufficiale per Python, la richiesta genera un'operazione di tipo `patch_namespaced_deployment_scale` sul Deployment `drone-simulator`, variando in tempo reale il numero di repliche a disposizione in base alla coda dei pacchi.

A tutela dell'infrastruttura (*Operational Safety*), lo scaling dinamico è sottoposto a policy di sicurezza:

```

1  # Verifica dei vincoli architetturali prima del patching su
    Kubernetes
2  if requested_replicas > POLICY_LIMITS:
3      # L'azione viene bloccata ed escalata all'operatore umano
4      trigger_human_escalation(requested_replicas)
5  else:
6      # Esecuzione della patch sul Deployment
7      k8s_client.patch_namespaced_deployment_scale(name="drone-
        simulator", ...)

```

Listing 5.2: Guardrail di sicurezza applicato alle richieste di scaling dell'agente.

Se l'agente tenta un'espansione del numero di repliche oltre la soglia prestabilita `POLICY_LIMITS` (vedi Capitolo 4), l'azione viene intercettata, bloccata ed escalata al *Human Approval Manager*, evitando consumi cloud incontrollati dovuti a potenziali allucinazioni o comportamenti imprevedibili del modello.

Capitolo 6

Chaos Engineering e Test di Fallimento

Per validare empiricamente i meccanismi di reliability descritti nel Capitolo 5, è stata realizzata una suite di chaos engineering (`ops/chaos_test_suite.py`) che inietta in modo controllato guasti rappresentativi sui componenti più critici dell'architettura (database, broker, capacità di flotta). L'obiettivo non è dimostrare l'assenza di guasti, ma misurare la *reazione* del sistema in termini di RTO, perdita di dati e capacità di auto-rimediazione agentica.

6.1 Metodologia

Ogni esperimento è strutturato secondo il ciclo classico del chaos engineering:

1. **Ipotesi:** definizione del comportamento atteso (SLO da rispettare).
2. **Blast radius:** il guasto è confinato a un singolo Deployment nel namespace `default`; il control-plane non viene toccato.
3. **Iniezione:** eseguita via API Kubernetes (`patch_namespaced_deployment_scale`, `delete_namespaced_pod`) o tramite generatore di carico (`client_simulator.py --stress`).
4. **Osservazione:** raccolta di metriche su `chaos_test_results.log` (timestamp, RTO, repliche prima/dopo).
5. **Abort condition:** timeout massimo di 60 s per il ripristino, oltre il quale il test è considerato fallito.

6.2 Esperimento 1: Primary Database Outage

Ipotesi. Lo spegnimento di InfluxDB non deve interrompere il monitoraggio real-time (*Graceful Degradation*, §5.2); la dashboard deve restare reattiva e il sistema deve recuperare entro l'SLO di disponibilità del data layer (§7.1).

Procedura. Scaling del Deployment `influxdb` a 0 repliche, attesa di 10 s, ripristino a 1 replica e cronometraggio del tempo necessario affinché il Pod torni in stato `Ready`.

Risultato. RTO misurato: 12,05 s (run del 23/05/2026). Durante l’outage il `central_server` ha continuato a servire la dashboard sfruttando lo stato in-memory; le scritture storiche sono state perse, in linea con il pattern di dual-write neutralizzato da `try/except`.

6.3 Esperimento 2: Network Resilience (MQTT Broker Crash)

Ipotesi. L’eliminazione forzata del Pod `mosquitto` non deve compromettere la flotta: Kubernetes deve ricreare il Pod e i client `paho-mqtt` devono riconnettersi autonomamente tramite *exponential backoff* (§5.1).

Procedura. Deletion del Pod via `delete_namespaced_pod` e attesa del nuovo Pod in stato `Running`.

Risultato. Downtime di scheduling Kubernetes: 0,04 s (rischedulazione immediata sul nodo). La riconnessione dei droni è gestita dal client MQTT senza logica custom; il broker non perde messaggi grazie alla natura *at-most-once* della telemetria (perdite tollerate, frequenza 2 s).

6.4 Esperimento 3: Load Spike e Scaling Agentico

Ipotesi. L’iniezione simultanea di 50 ordini deve essere rilevata dal `triage_manager` e tradursi in una richiesta di scaling dell’`HealthAgent`; sotto la soglia di 6 droni lo scaling avviene in autonomia, oltre tale soglia parte l’escalation HITL (§4.5).

Procedura. Lettura delle repliche correnti del Deployment `drone-simulator`, esecuzione di `client_simulator.py --stress` (durata effettiva ≈ 56 s), attesa ulteriore di 30 s per consentire al `LogisticAIBrain` di completare un ciclo di triage, riletture delle repliche.

Risultato osservato. Repliche **prima = 3, dopo = 6**. L’orchestratore agentico ha rilevato il backlog tramite query InfluxDB, invocato il tool MCP `scale_drone_fleet` e raddoppiato la flotta entro ≈ 86 s end-to-end dalla prima iniezione. Il test conferma l’efficacia dello scaling agentico per shock di carico controllati e mostra che le ottimizzazioni di latenza del ciclo decisionale (`time.sleep(30)` + inferenza LLM) sono sufficienti per orizzonti dell’ordine del minuto. Per shock sub-minuto resta aperta la traccia (§6.6) di un HPA Kubernetes nativo come *fast-path* complementare.

6.5 Riepilogo Metriche

Esperimento	Metrica	Valore
InfluxDB outage	RTO	12,05 s
MQTT broker crash	Downtime K8s	0,04 s
Load spike (50 ord.)	Δ repliche	$3 \rightarrow 6$ (≈ 86 s)

Tabella 6.1: Risultati misurati eseguendo `chaos_test_suite.py` (run del 23/05/2026).

6.6 Lezioni Apprese

Gli esperimenti confermano la robustezza dei livelli infrastrutturali (Kubernetes + `paho-mqtt` + dual-write) e validano lo strato agentico come meccanismo di scaling efficace su orizzonti di ≈ 1 min. Restano due limiti aperti:

1. la latenza del ciclo decisionale (≈ 30 s dominati da `time.sleep(30)` e inferenza LLM) è ancora elevata per spike sub-minuto in cui un HPA Kubernetes nativo, basato su metriche di code/CPU, offrirebbe un *fast-path* complementare;
2. la suite non misura ancora la *loss rate* di telemetria durante l'outage di InfluxDB né la latenza end-to-end di assegnazione degli ordini sotto stress.

La convivenza HPA + agente (HPA come reattore rapido, agente come pianificatore strategico) rientra tra gli sviluppi futuri discussi nel Capitolo 8.

Capitolo 7

Analisi Economica: Costi, ROI e ROA (Return on Agent)

7.1 Service Level Objectives (SLO) e Costo del Downtime

Definizione degli SLO

Per analizzare l'affidabilità del sistema e definirne la tolleranza ai guasti, sono stati stabiliti tre *Service Level Objectives* (SLO), coerenti con il workload presentato nei capitoli precedenti:

- **SLO 1 – Latenza di Assegnazione Logistica:** il 90% degli ordini ad alta priorità deve essere processato e assegnato a un mezzo entro 1 minuto. Il valore è giustificato dal ciclo di vita dell'orchestratore, che prevede un `time.sleep(30)` di base a cui si somma la latenza di inferenza dell'API.
- **SLO 2 – Disponibilità del Data Layer:** il broker MQTT e il database InfluxDB devono garantire il 99,9% di uptime. Considerato che i droni inviano telemetria ogni 2 secondi, una perdita prolungata di dati (per quanto mitigata dalle tecniche viste nel Capitolo 5) renderebbe cieca l'IA, paralizzando l'`HealthAgent` nella sua funzione di monitoraggio con possibili risvolti pericolosi.
- **SLO 3 – Affidabilità e risoluzione agentica:** l'LLM deve concludere il proprio task di ragionamento ed emettere una *tool call* valida nel 99% dei casi, senza innescare l'interruttore di sicurezza `MAX_AGENT_ITERATIONS`, evitando timeout e spreco di computazione.

Stima del Costo di un'Ora di Downtime

Operando nel dominio della logistica critica, il costo stimato di un'ora di downtime del servizio è di circa **15.000 €/ora**. La stima deriva da una media dei guadagni che il servizio raggiunge in un'ora di attività portando a termine le consegne, integrata dalle possibili penali contrattuali per violazione degli SLA.

7.2 Valutazione del Return on Agent (ROA) e Costi

L'orchestrazione autonoma della flotta introduce un bilanciamento critico tra i costi operativi delle API (OPEX) e il risparmio di capitale hardware (CAPEX) ottenuto tramite logiche predittive.

7.2.1 Strategie di Ottimizzazione dei Costi Operativi (OPEX)

Il sistema limita l'impatto economico delle chiamate LLM attraverso tre strategie integrate:

- **Triage preventivo:** il modulo `triage_manager` analizza lo snapshot del sistema prima di invocare i modelli, attivando `LogisticAgent` ed `HealthAgent` solo se sussistono reali condizioni di sbilanciamento (es. ordini pendenti accoppiati a droni idle, o necessità di scaling/manutenzione). Questo approccio azzerà i costi derivanti da cicli di no-op ridondanti.
- **Ingegneria dei prompt:** i prompt di sistema impongono restrizioni sintattiche che vietano la generazione di testo libero, preamboli o spiegazioni discorsive del ragionamento. L'output dei modelli è confinato alle sole chiamate di funzione (*tool calling*), minimizzando drasticamente il consumo di token di completamento.
- **Monitoraggio dei consumi:** il sistema esegue un auditing sistematico dei token spesi (prompt, completamento e totale) al termine di ogni esecuzione, permettendo una precisa rendicontazione finanziaria del processo decisionale.

7.2.2 Mitigazione del Rischio e Preservazione del Capitale (CAPEX)

Il ritorno sull'investimento dell'agente intelligente si concretizza nella mitigazione dei rischi d'impresa e nell'ottimizzazione infrastrutturale:

- **Manutenzione preventiva dinamica:** il `LogisticAgent` include nei prompt operativi la regola di evitare l'assegnazione di carichi pesanti ($> 3\text{kg}$) a droni con usura superiore al 70%. Questa policy, pur essendo oggi principalmente prompt-driven (*soft constraint*), orienta il matching ordine-drone verso scelte conservative che riducono il rischio di esaurimento batteria e perdita del vettore.
- **Elasticità infrastrutturale cloud:** l'`HealthAgent` modula dinamicamente le repliche del `Deployment drone-simulator` su Kubernetes basandosi sulle fluttuazioni del carico di lavoro effettivo, riducendo i costi di over-provisioning dell'infrastruttura cloud.
- **Operational Safety Cap:** le policy di sicurezza impongono un limite massimo di 6 droni per lo scaling automatico. Qualsiasi espansione ulteriore viene intercettata e subordinata ad approvazione umana (`request_human_approval`), ponendo un vincolo rigido (*budget cap*) contro anomalie o loop di scaling incontrollati.

- **Massimizzazione del throughput:** la coda degli ordini viene ordinata preventivamente per priorità. L’agente logistico alloca la capacità di carico residua privilegiando i task *high priority*, lasciando il valore economico come criterio secondario nel ragionamento del modello.

Proiezioni Economiche e Budget Compliance

Per convalidare la sostenibilità finanziaria del software è stata effettuata una stima comparativa dei costi su base mensile, ipotizzando una frequenza di esecuzione standard di 2.880 cicli giornalieri (un polling di verifica ogni 30 secondi).

Voce di Costo	Gemini 2.5 Pro	Mistral Small
Infrastruttura Cloud (1 control-plane + 2 worker)	150,00 \$ / mese	150,00 \$ / mese
Token Modello Linguistico (AI)	~500,00 \$ / mese	~10,00 \$ / mese
OPEX Totale Mensile	~650,00 \$ / mese	~160,00 \$ / mese

Tabella 7.1: Confronto dei costi mensili al variare del modello LLM utilizzato.

- **Costi infrastrutturali fissi:** il mantenimento dell’ambiente containerizzato in alta affidabilità (comprensivo di 1 control-plane e 2 nodi worker Kubernetes, database InfluxDB per la telemetria e broker MQTT per la messaggistica) genera un costo operativo fisso di circa **150 \$/mese**.
- **Costi variabili dell’Intelligenza Artificiale:** l’impatto della componente generativa è fortemente condizionato dall’architettura del modello scelto. *Gemini 2.5 Pro* comporta un volume medio di 1.200 token per ciclo (ripartiti equamente tra input e output), traducendosi in una spesa di circa **500 \$/mese**. Un modello specialistico più leggero come *Mistral Small* riduce l’impronta a 800 token per ciclo; grazie a un’architettura asimmetrica focalizzata sull’input (90% input, 10% output), il costo computazionale si abbatte a meno di **10 \$/mese**.

L’analisi finanziaria evidenzia che la quasi totalità del costo variabile è legata al modello linguistico sottostante. Mentre l’infrastruttura cloud mantiene un costo fisso e invariabile di 150\$/mese, la spesa per l’intelligenza artificiale fluttua significativamente, generando una marcata **forbice dei costi** che spazia dai 500\$/mese di *Gemini 2.5 Pro* a meno di 10\$/mese per *Mistral Small*. Questo divario si traduce in un delta operativo di circa **490 \$ mensili**, offrendo la possibilità di modulare la scelta dell’LLM in base al livello di complessità decisionale richiesto in tempo reale.

Per scenari ordinari di instradamento degli ordini e scaling lineare, la flotta può essere orchestrata efficacemente da modelli specialistici e leggeri, riducendo l’OPEX totale ad appena **160 \$/mese**. Al contrario, l’allocazione del modello top-tier può essere riservata esclusivamente a finestre temporali critiche caratterizzate da anomalie infrastrutturali complesse, elevata congestione della coda o conflitti di policy che richiedono capacità di ragionamento superiori.

7.3 Analisi dei Trade-Off

1. Trade-off tra Costo e Affidabilità

L'adozione di un sistema basato su agenti LLM introduce un'inversione di paradigma rispetto a un sistema deterministico tradizionale, ridefinendo il concetto stesso di continuità operativa:

- **Sistema deterministico:** presenta costi di computazione (OPEX) minimi, limitati all'esecuzione di script locali. Il limite principale risiede nella resilienza operativa: a fronte di eccezioni non gestite o anomalie di rete, i flussi tradizionali subiscono arresti critici. Il ripristino del servizio richiede l'intervento manuale di un amministratore di sistema, incrementando i costi legati alla reperibilità oraria e introducendo latenze nei tempi di risoluzione.
- **Sistema multi-agente:** introduce un costo operativo variabile legato al consumo di token delle API (stimato fino a 500 \$/mese con *Gemini 2.5 Pro*). Questo investimento si traduce in una migliore gestione degli imprevisti grazie alla flessibilità semantica del modello, che gli consente di contestualizzare l'errore, ragionare sulla deviazione e deviare dinamicamente su flussi alternativi, garantendo la continuità del servizio senza richiedere l'intervento umano.

2. Trade-off tra Automazione e Sicurezza (Automation vs. Safety)

Questo bilanciamento definisce le soglie di autonomia decisionale del software prima della transizione del controllo all'operatore umano:

- **Sistema deterministico:** garantisce l'osservanza rigorosa delle policy di sicurezza e dei vincoli operativi grazie a logiche cablate nel codice. Tale rigidità ne limita tuttavia la flessibilità: a fronte di anomalie o picchi improvvisi di carico (come la sovrapposizione tra l'accumulo di ordini prioritari e il fermo contemporaneo di più vettori) l'algoritmo non è in grado di rimodulare autonomamente le soglie di tolleranza o di deviare dai flussi standard, provocando il blocco dei processi logistici fino a un intervento manuale.
- **Sistema multi-agente:** massimizza l'efficienza dei processi decisionali, abilitando gli agenti a valutare trade-off complessi tra metriche di business (ottimizzazione dei ricavi) e vincoli fisici (usura dei vettori). Di contro, l'elasticità decisionale introduce rischi legati alla sicurezza deterministica, quali potenziali allucinazioni nell'invocazione dei tool o derive logiche nelle richieste di scaling infrastrutturale, mitigati dai guardrail descritti nel Capitolo 4.

Capitolo 8

Conclusioni

Il progetto ha dimostrato la fattibilità tecnica di un'architettura logistica resiliente in cui il livello di automazione viene affidato a un layer di *Agentic AI* mediato da un server MCP, con l'orchestratore (`logistic_ai_brain.py`) che funge da *triage manager* e *context router* per due agenti specializzati: l'**HealthAgent** sul piano infrastrutturale e il **LogisticAgent** sul piano di business. La separazione tra LLM, layer MCP e infrastruttura Kubernetes ha permesso di soddisfare contemporaneamente i requisiti di osservabilità, sicurezza e scalabilità discussi nei capitoli precedenti.

8.1 Sintesi dei Risultati

I principali risultati raggiunti possono essere riassunti su tre piani complementari:

- **Reliability infrastrutturale:** i meccanismi di *graceful degradation* del `central_server`, di *exponential backoff* di `paho-mqtt` e di rischedulazione nativa di Kubernetes hanno garantito, negli esperimenti di chaos engineering (Capitolo 6), un RTO contenuto (circa 12s sull'outage di InfluxDB) e una rischedulazione pressoché immediata del broker MQTT, senza interruzione del monitoraggio real-time.
- **Safety agentica:** l'adozione del principio del *Least Privilege*, dei contatori di iterazione, del *kill-switch* sul backend e del paradigma HITL ha confinato l'autonomia dell'LLM entro confini verificabili. La soglia architetturale dei 6 droni per lo scaling automatico ha permesso di mantenere il controllo umano sulle azioni a maggiore impatto economico.
- **Sostenibilità economica:** l'analisi di costo (Capitolo 7) ha quantificato la *forbice* tra modelli LLM differenti e ha evidenziato come il sistema, grazie al *triage manager* e all'idempotenza dei tool, mantenga l'OPEX in un intervallo prevedibile (160–650 \$/mese a seconda del modello), abilitando una scelta consapevole tra capacità di ragionamento e costo per ciclo.

8.2 Limitazioni Attuali

L'analisi sperimentale ha messo in luce alcune limitazioni dell'approccio attuale:

- **Copertura dei test di caos:** la suite attuale non misura ancora la *loss rate* di telemetria durante l’outage di InfluxDB né la latenza end-to-end di assegnazione degli ordini sotto stress; i risultati sono quindi conservativi e parziali.
- **Stima economica:** i valori di OPEX (Tabella 7.1) e il costo orario di downtime di 15.000 € sono basati su ipotesi di carico costante e su assunzioni di mercato; in produzione richiederebbero una validazione su dati reali della flotta.

8.3 Sviluppi Futuri

A partire dalle limitazioni identificate, il progetto si presta a diverse linee evolutive coerenti con l’impianto architetturale esistente:

1. **Routing dinamico tra modelli LLM:** sfruttare la forbice di costo evidenziata nel Capitolo 7 implementando un meccanismo che selezioni dinamicamente il modello (es. *Mistral Small* per cicli ordinari, *Gemini 2.5 Pro* per anomalie complesse), ottimizzando ulteriormente il *Return on Agent*.
2. **Validazione su dispositivi fisici:** estendere il digital twin con un’integrazione verso droni reali o simulatori ad alta fedeltà, per verificare la trasferibilità del modello fisico (velocità, batteria, usura) oltre l’ambiente simulato.

In definitiva, il sistema realizzato dimostra come l’introduzione di un layer agentico mediato da MCP, accompagnato da guardrail di sicurezza e da una rigorosa strategia di osservabilità, possa elevare il livello di automazione di un’infrastruttura logistica critica senza sacrificare la prevedibilità operativa e il controllo umano sulle decisioni ad alto impatto.